

Student Q&A Forum

COMP1841 Coursework Report

PHP/MySQL CRUD system for a student support forum

Website URL	http://localhost/COMP1841/CourseWork/
Admin login	admin@example.com / admin123
Database	comp1841_coursework
Word count	Approx. 1,829 words
Student name	[Add your name]
Student ID	[Add your ID]

Table of Contents

1 Introduction	3
2 System design	4
2.1 Page layout and navigation	4
2.2 Data model	5
2.3 Project organisation	5
3 Technologies used	6
4 Legal, social and ethical considerations	7
5 System overview and screenshots	9
6 Testing and validation	11
7 Conclusion and future recommendations	12
References	13

1 Introduction

The Student Q&A Forum is a compact prototype built for COMP1841 to help students share coursework questions in one place. The system supports public browsing, account creation, login, question posting, editing, deletion, module tagging, image attachments, and an administrator area for user and module maintenance. The application is intentionally small, but it follows the same basic structure as a real student support platform.

The functional scope maps directly to the coursework brief. Public visitors can browse questions, authenticated users can create and manage their own posts, and administrators can manage accounts and module names. A contact form is also included so students can send a message to the administrator, although the current prototype simulates email delivery rather than sending through SMTP.

This report explains the site design, database model, technology choices, security decisions, legal and ethical issues, and a short testing record. The discussion focuses on what is implemented in the current codebase rather than generic theory.

The prototype also shows how a modest codebase can still demonstrate proper engineering habits. Even without a framework, the use of templates, helper functions, and a compact schema keeps the flow readable and makes it easier to explain the design decisions in an academic report.

2 System design

2.1 Page layout and navigation

The project uses two shared layout templates: `templates/layout.html.php` for the public site and `templates/admin_layout.html.php` for the admin area. This keeps the site consistent while avoiding duplicated header and footer markup. The navigation bar changes according to session state so that guests see Login, Sign Up, and Contact, while logged-in users see Ask Question and Logout. Administrators also see the Admin Area link.

The public flow is intentionally short. Visitors land on Home, move to Questions, open a post to read the full content and image, and then either log in, sign up, or contact the administrator. Logged-in users can add or edit questions through the same form, which reduces duplication and keeps the create/edit workflow aligned.

2.2 Data model

The database is intentionally compact but relational. `accounts` stores user identity, hashed password, role, display name, and bio. `post` stores each question and references the author through `author_id`. `category` stores module names, and `post_category` acts as a junction table so that one post can belong to multiple modules while each module can be reused across many posts.

Foreign keys enforce referential integrity, and cascading deletes keep the tables consistent when a post or account is removed. Unique constraints on `username`, `email`, and `category.name` prevent duplicate records. This structure is more robust than storing module names directly inside the `post` table because it supports filtering, reuse, and cleaner updates.

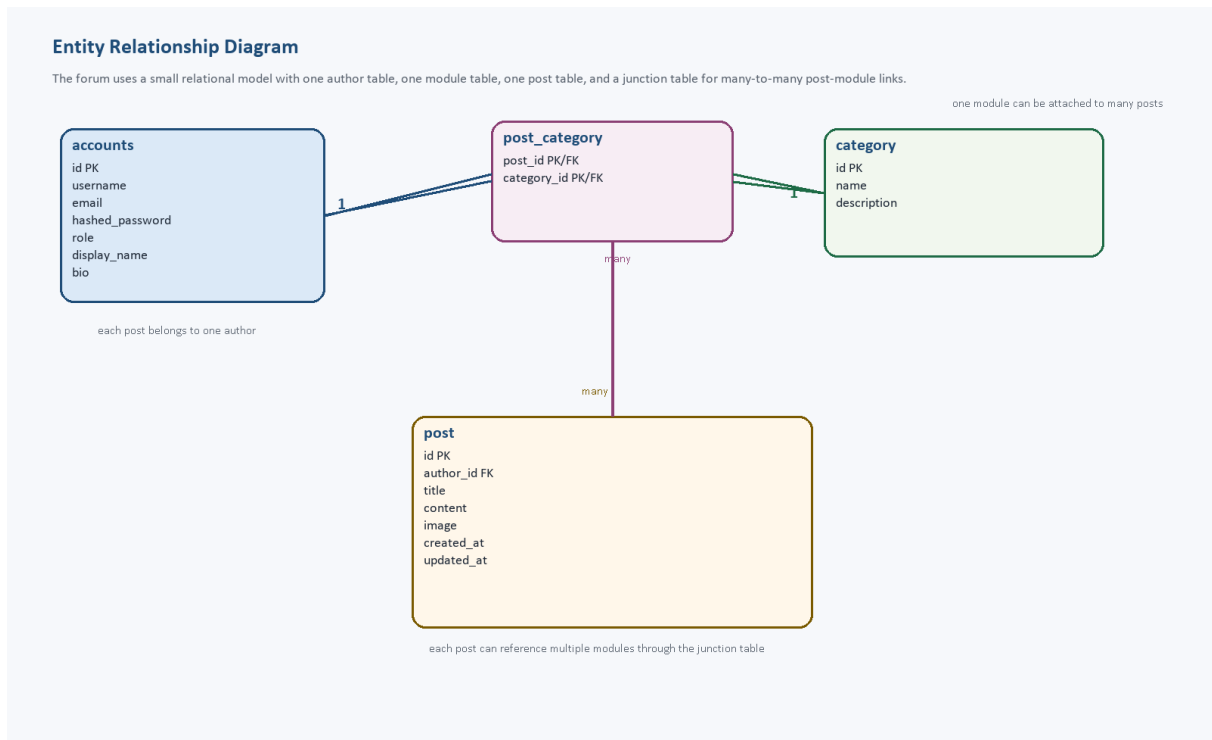


Figure 1. Entity relationship diagram for accounts, post, category, and the post_category junction table.

2.3 Project organisation

The codebase is split by responsibility. Root-level PHP files handle page entry points, includes/ contains the database connection and reusable data functions, templates/ contains the presentation layer, admin/ contains protected management pages, img/ stores source assets, and uploads/ stores user-submitted screenshots with generated filenames. Setup scripts and schema files live in scratch/.

This separation makes the system easier to maintain because HTML structure, data access, and business rules stay in different places. The post form and listing pages can reuse the same helper functions, and the admin area can reuse the same account and category functions without copying SQL into multiple files.

Project structure overview:

index.php / posts.php / post_view.php	Public entry points for the home page, question listing, and single-post view.
post_action.php / post_delete.php	Create, update, and delete logic for question posts.
login.php / signup.php / logout.php	Authentication and session lifecycle.
contact.php	Prototype contact form for administrator messages.
includes/	Database connection plus reusable query and CRUD helper functions.
templates/	Shared HTML/PHP view templates for the public and admin layouts.
admin/	Protected admin dashboard, user management, and module management.
uploads/	Stored screenshot uploads with generated unique filenames.

3 Technologies used

PHP is used as the server-side language because the coursework requires PHP/MySQL and because PHP works naturally with the existing XAMPP stack. Database calls use PDO and prepared statements rather than string-concatenated SQL, which keeps the code clearer and reduces injection risk (PHP, n.d.-c).

MySQL with InnoDB provides the relational storage layer. InnoDB foreign keys support the author and module relationships, and the schema is small enough to be understood quickly while still demonstrating proper normalization. The front end uses HTML5 forms and a modest CSS stylesheet, with semantic labels, required fields, email validation, and a responsive layout that collapses on smaller screens.

Session handling supports authentication and role checks. The login flow stores only session identifiers and non-sensitive user details, while passwords are validated with `password_verify()` against values created by `password_hash()` (PHP, n.d.-a; PHP, n.d.-b). File uploads are handled through `move_uploaded_file()` and a filename strategy that combines randomness with a timestamp (PHP, n.d.-d).

These choices suit a coursework prototype because they are simple, transparent, and easy to inspect. The implementation stays close to the underlying technologies instead of hiding them behind a framework, which makes the database, sessions, and request handling easier to explain in the report.

The absence of a framework is deliberate. Using direct PHP pages and shared templates makes the request flow easy to trace from URL to database call to rendered HTML. That is useful in coursework because the logic remains visible and the report can reference concrete files rather than framework conventions.

4 Legal, social and ethical considerations

The main personal data stored by the forum is limited to username, email address, display name, biography, hashed password, and uploaded screenshots. That is enough to support account management and question ownership without collecting unnecessary information. The design follows data-minimisation principles by avoiding real names, phone numbers, or address fields. That matters because student forum data can become sensitive when it is linked to coursework activity or identity (ICO, n.d.).

Password handling is a key security issue. Plain-text passwords would expose users if the database were leaked, so the system stores only one-way hashes and never writes the original password back to disk (PHP, n.d.-a; PHP, n.d.-b). Prepared statements are also used to reduce the risk of SQL injection, and role-based checks prevent ordinary users from entering the admin area or deleting posts they do not own.

The image upload flow is restricted to a small whitelist of image extensions, and filenames are generated uniquely rather than reusing the original upload name. This avoids accidental overwriting and makes it harder to infer the source device or filename history. A production version should go further by checking MIME type, file size, virus scanning, and image dimensions before storage (OWASP Foundation, n.d.).

Accessibility is handled at a basic level through real form labels, readable text contrast, keyboard-accessible links, and HTML5 validation. The current interface is intentionally simple, which helps screen readers and reduces interaction complexity, but it would still benefit from a fuller accessibility pass against WCAG guidance, especially for colour contrast, mobile spacing, and clearer error summaries (W3C, n.d.).

From a social and ethical perspective, the forum needs light moderation. A student support site can be useful only if posts remain respectful and personal data is not overshared. The current admin area gives the lecturer or administrator a control point for users and modules, and the contact form provides a low-friction route for issues without exposing the administrator's private email address. In a live deployment, the prototype email action should be replaced with proper SMTP delivery and logging so messages are not lost.

Although the project is local, GDPR thinking still matters. UK GDPR and the Data Protection Act 2018 continue to shape how personal data should be collected, stored, and deleted after Brexit. A real deployment would need a lawful basis for storing account data, a retention policy for old questions and uploaded screenshots, and a way to export or delete a user's data on request (ICO, n.d.).

Data retention also matters. Student questions and uploaded screenshots may contain names, code snippets, or course details, so a production system would need a retention schedule and deletion workflow. Backups should be encrypted and access controlled, and old attachments should not be kept forever without a clear reason (ICO, n.d.).

Accountability is another legal concern. The administrator area gives a single trusted role control over users and modules, but that role should be limited to people who genuinely need it. A real deployment would benefit from audit logging of major actions such as role changes, deletions, and contact messages

so incidents can be investigated later. The current prototype keeps the admin surface small, which is sensible for a coursework system, but the same code path would need stricter oversight in production.

The contact form is also deliberately narrow: it requests only the minimum information needed to reply. That is important because a coursework forum should not encourage students to submit student numbers, phone numbers, or other extra identifiers unless there is a specific need. A live version would usually pair the form with a privacy notice and spam protection.

If a user asks for deletion or correction, the data model makes that manageable because personal information is stored in accounts and question records are linked through foreign keys. That structure means a future admin tool could export or remove a user's activity without flattening the whole database, which is much safer than storing everything in one table.

5 System overview and screenshots

5.1 Public workflow

Figure 3 shows the public workflow. The Questions page supports filtering by module through a dropdown query, so students can narrow the list to a single course area. Opening a question displays the full text and any attached image, which is important for coursework screenshots and debugging contexts. The Ask Question form allows multiple checkboxes and an optional screenshot upload, so one post can be linked to more than one module.

The Contact Administrator page sits beside the public workflow and gives visitors a simple way to raise an issue. In the current prototype, form submission shows a success message rather than sending a real email, which keeps the coursework focused on web flow rather than mail server configuration.

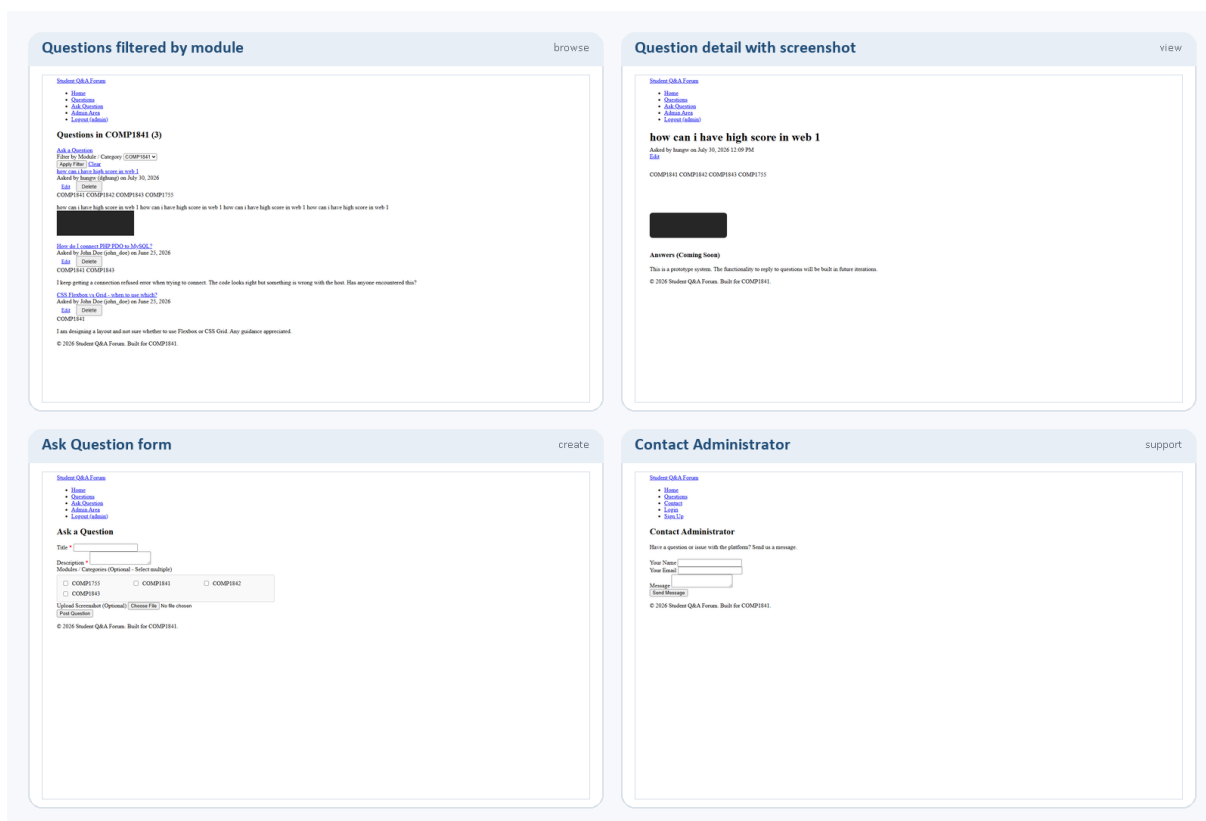


Figure 3. Public workflow covering module filtering, question detail, posting, and the contact form.

5.2 Admin workflow

Figure 4 shows the authenticated workflow. The login screen uses email and password fields, and after session creation the admin dashboard exposes user and module management. The users screen lets the administrator change roles or delete accounts, and the modules screen allows new module names to be added or existing ones removed.

These screens prove the role-based controls in the code. Ordinary users cannot reach the admin pages, while the navigation bar exposes Admin Area only after login with an administrator session. That keeps

the administrative tools separate from the student-facing content while still using the same overall theme and data layer.

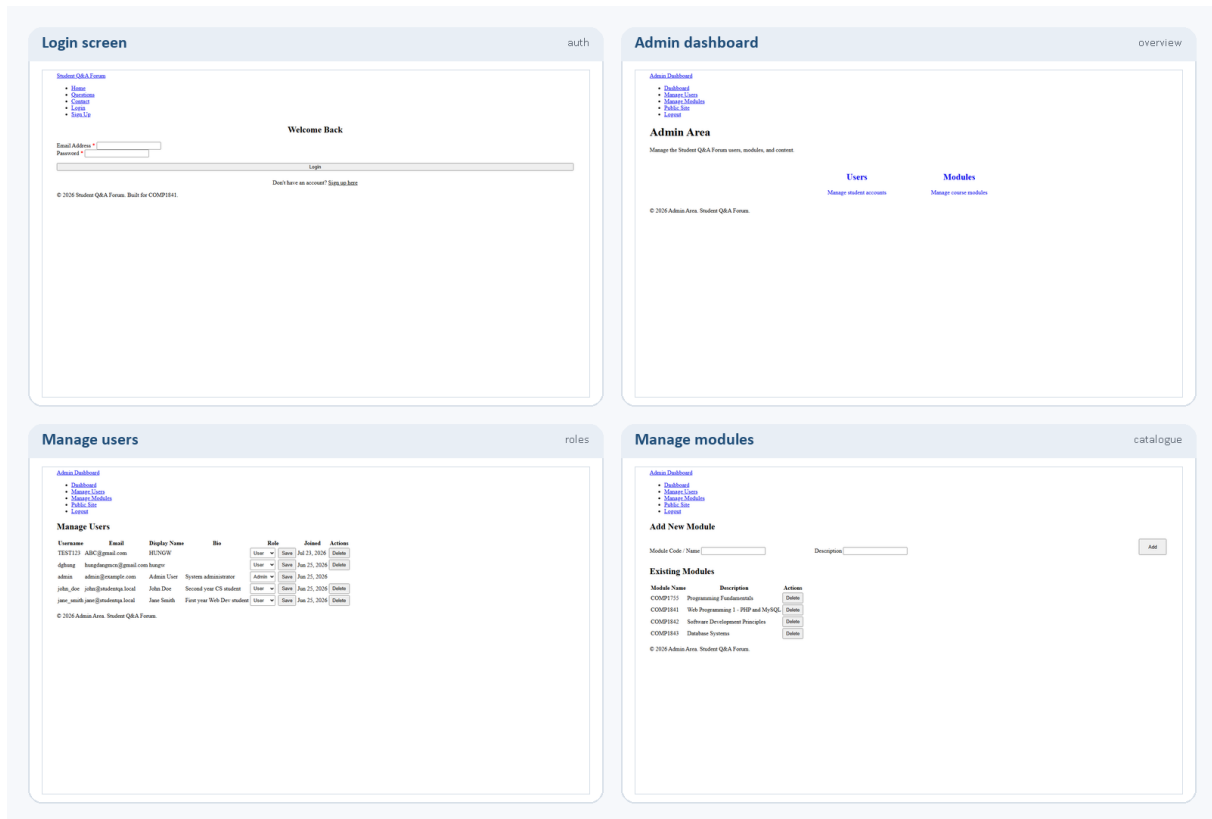


Figure 4. Admin workflow covering login, the dashboard, user management, and module maintenance.

6 Testing and validation

Testing was carried out as a structured mix of positive and negative cases. The goal was to confirm that the main user journeys worked, that invalid input was blocked cleanly, and that permissions stopped users from crossing role boundaries. The results below focus on the functions that matter most for the rubric.

ID	Test case	Input / action	Expected result	Result
T1	Sign up with unique username, email, and password	Complete the registration form	Account is created and the user is redirected to Login	Pass
T2	Attempt sign up using a duplicate username or email	Reuse an existing account value	Validation error is shown and the insert is blocked	Pass
T3	Log in with valid administrator credentials	admin@example.com / admin123	Session is created and the dashboard becomes available	Pass
T4	Log in with an incorrect password	Enter a wrong password	The system shows an invalid login message	Pass
T5	Create a question with categories and an image	Use the Ask Question form	Question is saved and the upload receives a unique filename	Pass
T6	Upload an invalid file type	Choose a non-image file	The upload is rejected with an error message	Pass
T7	Edit or delete a question as the owner	Open the question while logged in	The post can be updated or removed	Pass
T8	Open another user's question as a standard user	Try to edit or delete another user's post	The action is blocked unless the session is admin	Pass
T9	Filter questions by COMP1841	Use the module dropdown	Only matching posts are listed	Pass
T10	Open admin pages without an administrator session	Visit admin/index.php directly	Access is denied or redirected to the public site	Pass
T11	Submit the contact form	Fill the form and press Send Message	A confirmation message is shown	Pass (prototype email)

All core cases passed in the current build. The only deliberate limitation is the contact form, which confirms submission but does not yet send through a live mail server. That is acceptable for a prototype, but it should be replaced before any public deployment.

7 Conclusion and future recommendations

The project meets the core COMP1841 CRUD requirements and also adds login, signup, password hashing, role-based admin access, image uploads with unique filenames, and a clear template-based structure. The design is still prototype-level, but the main workflows are complete and the code is organised enough to extend.

Future improvements should focus on replacing the simulated contact action with real SMTP delivery, adding CSRF tokens, improving upload validation, introducing pagination and search, expanding the answer/reply feature, and tightening the mobile layout. Those changes would move the project from coursework prototype to a more realistic student support platform.

Future work should prioritise:

1. Replace the simulated contact action with real SMTP delivery.
2. Add CSRF protection and stricter MIME / size checks for uploads.
3. Introduce pagination, search, and an answer or reply feature.
4. Improve the mobile layout and accessibility audit.
5. Add edit screens for modules and richer user profile management.

References

ICO (n.d.) UK GDPR guidance and resources. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/> (Accessed: 30 July 2026).

MDN Web Docs (n.d.) Client-side form validation. Available at: https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation (Accessed: 30 July 2026).

OWASP Foundation (n.d.) File Upload Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (Accessed: 30 July 2026).

PHP (n.d.-a) password_hash. Available at: <https://www.php.net/manual/en/function.password-hash.php> (Accessed: 30 July 2026).

PHP (n.d.-b) password_verify. Available at: <https://www.php.net/manual/en/function.password-verify.php> (Accessed: 30 July 2026).

PHP (n.d.-c) PDO::prepare. Available at: <https://www.php.net/manual/en/pdo.prepare.php> (Accessed: 30 July 2026).

PHP (n.d.-d) move_uploaded_file. Available at: <https://www.php.net/manual/en/function.move-uploaded-file.php> (Accessed: 30 July 2026).

W3C (n.d.) Accessibility fundamentals. Available at: <https://www.w3.org/WAI/fundamentals/accessibility-intro/> (Accessed: 30 July 2026).